

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 5

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2026

1 Задача

Реализовать один из алгоритмов отсечения определенного типа в пространстве заданной размерности (в соответствии с вариантом).

- Сазерленда-Ходжмена
- Двумерное
- Внешнее

Ввод исходных данных каждого из алгоритмов производится интерактивно с помощью клавиатуры и/или мыши (за исключением особо оговоренных случаев).

2 Основная теория

2.1 Отсечение многоугольников

Отсечение (clipping) — это процесс выделения части геометрического объекта, попадающей в заданную область (окно отсечения). Для двумерных многоугольников одной из классических задач является отсечение произвольного многоугольника выпуклым отсекающим многоугольником. Алгоритм Сазерленда–Ходжмена решает эту задачу эффективно и просто.

2.2 Алгоритм Сазерленда–Ходжмена

Алгоритм работает с выпуклым отсекающим многоугольником, заданным упорядоченным списком вершин. Исходный отсекаемый многоугольник также задаётся списком вершин. Принцип заключается в последовательной обработке каждого ребра отсекателя.

Пусть текущее ребро отсекателя задано точками A и B. Для каждого ребра SP текущего многоугольника (изначально — исходный многоугольник) проверяется, лежат ли точки S и P относительно прямой AB со стороны, считающейся «внутренней». Для этого вычисляется знак ориентированной площади (векторное произведение):

$$R = (B_x - A_x)(Q_y - A_y) - (B_y - A_y)(Q_x - A_x)$$

Если $R \geq 0$, то точка Q лежит слева от направленного отрезка AB; при обходе отсекателя против часовой стрелки область слева считается внутренней.

Для каждого ребра SP возможны четыре случая:

1. S снаружи, P внутри: добавляется точка пересечения SP с AB, затем P.
2. S внутри, P внутри: добавляется P.
3. S внутри, P снаружи: добавляется точка пересечения.
4. S снаружи, P снаружи: ничего не добавляется.

Точка пересечения отрезков SP и AB находится аналитически решением системы линейных уравнений. После обработки всех рёбер отсекаемого многоугольника для одного ребра отсекателя формируется новый промежуточный многоугольник. Затем алгоритм переходит к следующему ребру отсекателя. В конце получается многоугольник, являющийся пересечением исходного многоугольника и отсекателя (внутренняя область).

3 Практическая реализация

Листинг 1: Основной код

```
import glfw
from OpenGL.GL import *
import numpy as np

WINDOW_WIDTH = 800
WINDOW_HEIGHT = 600
IS_CLIP_POLYGON_FINISHED = False

class Polygon():
    def __init__(self, buffer, edges_color=(255, 0, 0), fill_color =(200, 200, 200)):
        self.points = []
        self.edges_color = edges_color
        self.fill_color = fill_color
        self.buffer = buffer

```

```
16      def clear_buffer(self):
17          self.buffer.fill(255)
18          self.points = []
19
20      def add_point(self, x, y):
21          self.points.append((x, y))
22
23      def set_pixel(self, x, y, color):
24          if 0 <= x < self.buffer.shape[1] and 0 <= y < self.buffer.shape[0]:
25              self.buffer[y, x] = color
26
27      def is_edge_point(self, x, y):
28          h, w = self.buffer.shape[:2]
29          return not (0 <= y < h and 0 <= x < w) or tuple(self.buffer[y, x]) == self.edges_color
30
31      def is_filled_point(self, x, y):
32          h, w = self.buffer.shape[:2]
33          return 0 <= y < h and 0 <= x < w and tuple(self.buffer[y, x]) == self.fill_color
34
35      def is_free(self, x, y, x_left):
36          return not self.is_edge_point(x, y) and not self.is_filled_point(x, y) \
37                      and (x == x_left or self.is_edge_point(x - 1, y) or self.is_filled_point(x -
    1, y))
38
39      def bresenham(self, point1, point2):
40          x0, y0 = map(int, point1)
41          x1, y1 = map(int, point2)
42          dx = abs(x1 - x0)
43          dy = abs(y1 - y0)
44          sx = 1 if x0 < x1 else -1
45          sy = 1 if y0 < y1 else -1
46
47          if dx >= dy:
48              e = -dx
49              de = 2 * dy
50              while True:
51                  self.set_pixel(x0, y0, self.edges_color)
52                  if x0 == x1:
53                      break
54                  e += de
55                  x0 += sx
56                  if e >= 0:
57                      y0 += sy
58                      e -= 2 * dx
59          else:
60              e = -dy
```

```
61                de = 2 * dx
62                while True:
63                    self.set_pixel(x0, y0, self.edges_color)
64                    if y0 == y1:
65                        break
66                    e += de
67                    y0 += sy
68                    if e >= 0:
69                        x0 += sx
70                        e -= 2 * dy
71
72        def fill_with_zatravka(self, x0, y0):
73            stack = [(x0, y0)]
74            while len(stack) != 0:
75                x, y = stack.pop()
76                x_left, x_right = x, x
77                while not self.is_edge_point(x_left, y):
78                    if tuple(self.buffer[y, x]) == (255, 255, 255):
79                        self.set_pixel(x_left, y, self.fill_color)
80                    x_left -= 1
81                while not self.is_edge_point(x_right, y):
82                    if tuple(self.buffer[y, x]) == (255, 255, 255):
83                        self.set_pixel(x_right, y, self.fill_color)
84                    x_right += 1
85                x_left += 1
86                x_right -= 1
87                for xi in range(x_left, x_right + 1):
88                    if self.is_free(xi, y - 1, x_left):
89                        stack.append((xi, y - 1))
90                    if self.is_free(xi, y + 1, x_left):
91                        stack.append((xi, y + 1))
92
93    buffer = np.full((WINDOW_HEIGHT, WINDOW_WIDTH, 3), (255, 255, 255), dtype=np.
          uint8)
94    clip_polygon = Polygon(buffer, edges_color=(255, 0, 0))
95    clipping_polygon = Polygon(buffer, edges_color=(0, 0, 255), fill_color=(200, 200, 255))
96
97    def main():
98        if not glfw.init():
99            return
100       window = glfw.create_window(WINDOW_WIDTH, WINDOW_HEIGHT, "Lab5", None,
          None)
101       if not window:
102           glfw.terminate()
103           return
104       glfw.make_context_current(window)
```

```
105     glfw.set_key_callback(window, key_callback)
106     glfw.set_mouse_button_callback(window, mouse_button_callback)
107 
108     glEnable(GL_TEXTURE_2D)
109     texture = glGenTextures(1)
110     while not glfw.window_should_close(window):
111         glfw.poll_events()
112 
113         glBindTexture(GL_TEXTURE_2D, texture)
114         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST
        )
115         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER,
       GL_NEAREST)
116         glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, buffer.shape[1], buffer.shape[0],
117                      0, GL_RGB, GL_UNSIGNED_BYTE, buffer)
118 
119         glClear(GL_COLOR_BUFFER_BIT)
120         glClearColor(1.0, 1.0, 1.0, 1.0)
121         glLoadIdentity()
122 
123         glBegin(GL_QUADS)
124         glTexCoord2f(0, 0); glVertex2f(-1, -1)
125         glTexCoord2f(1, 0); glVertex2f(1, -1)
126         glTexCoord2f(1, 1); glVertex2f(1, 1)
127         glTexCoord2f(0, 1); glVertex2f(-1, 1)
128         glEnd()
129 
130         glfw.swap_buffers(window)
131     glfw.destroy_window(window)
132     glfw.terminate()
133 
134 def orientation(a, b, c):
135     return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])
136 
137 def inside(p, a, b):
138     return orientation(a, b, p) >= 0
139 
140 def intersection(a1, a2, b1, b2):
141     x1, y1 = a1
142     x2, y2 = a2
143     x3, y3 = b1
144     x4, y4 = b2
145     d = (x1-x2)*(y3-y4) - (y1-y2)*(x3-x4)
146     if d == 0:
147         return None
148     px = ((x1*y2 - y1*x2)*(x3-x4) - (x1-x2)*(x3*y4 - y3*x4)) / d
```

```
149     py = ((x1*y2 - y1*x2)*(y3-y4) - (y1-y2)*(x3*y4 - y3*x4)) / d
150     return (px, py)
151
152 def Sutherland_Hodgman(subject, clip):
153     output = subject
154     for i in range(len(clip)):
155         input_list = output
156         output = []
157         print(input_list)
158         if not input_list:
159             return []
160         A = clip[i - 1]
161         B = clip[i]
162         S = input_list[-1]
163         for P in input_list:
164             if inside(P, A, B):
165                 if not inside(S, A, B):
166                     I = intersection(S, P, A, B)
167                     if I:
168                         output.append(I)
169                 output.append(P)
170             elif inside(S, A, B):
171                 I = intersection(S, P, A, B)
172                 if I:
173                     output.append(I)
174             S = P
175     return output
176
177 def key_callback(window, key, scancode, action, mods):
178     global IS_CLIP_POLYGON_FINISHED
179     if action == glfw.PRESS:
180         if key == glfw.KEY_C:
181             IS_CLIP_POLYGON_FINISHED = False
182             clip_polygon.clear_buffer()
183             clipping_polygon.clear_buffer()
184         if key == glfw.KEY_V:
185             buffer.fill(255)
186             result = Sutherland_Hodgman(clipping_polygon.points, clip_polygon.points)
187             clipping_polygon.points = result
188             for i in range(len(clip_polygon.points)):
189                 p1 = clip_polygon.points[i-1]
190                 p2 = clip_polygon.points[i]
191                 clip_polygon.bresenham(p1, p2)
192             for i in range(len(clipping_polygon.points)):
193                 p1 = clipping_polygon.points[i-1]
194                 p2 = clipping_polygon.points[i]
```

```
                    clipping_polygon.bresenham(p1, p2)
            if key == glfw.KEY_SPACE:
                if IS_CLIP_POLYGON_FINISHED:
                    if len(clipping_polygon.points) > 2:
                        clipping_polygon.bresenham(clipping_polygon.points[-1], clipping_polygon.
    points[0])
                else:
                    IS_CLIP_POLYGON_FINISHED = True
                    if len(clip_polygon.points) > 2:
                        clip_polygon.bresenham(clip_polygon.points[-1], clip_polygon.points[0])

def mouse_button_callback(window, button, action, mods):
    if action == glfw.PRESS:
        win_w, win_h = glfw.get_window_size(window)
        buf_w, buf_h = buffer.shape[1], buffer.shape[0]
        x, y = glfw.get_cursor_pos(window)
        x = int(x * buf_w / win_w)
        y = int(y * buf_h / win_h)
        y = buf_h - y
        if button == glfw.MOUSE_BUTTON_LEFT:
            if IS_CLIP_POLYGON_FINISHED:
                clipping_polygon.add_point(int(x), int(y))
                if len(clipping_polygon.points) > 1:
                    clipping_polygon.bresenham(clipping_polygon.points[-2], clipping_polygon.
    points[-1])
            else:
                clip_polygon.add_point(int(x), int(y))
                if len(clip_polygon.points) > 1:
                    clip_polygon.bresenham(clip_polygon.points[-2], clip_polygon.points[-1])
        if button == glfw.MOUSE_BUTTON_RIGHT:
            clipping_polygon.fill_with_zatravka(int(x),int(y))


main()
```

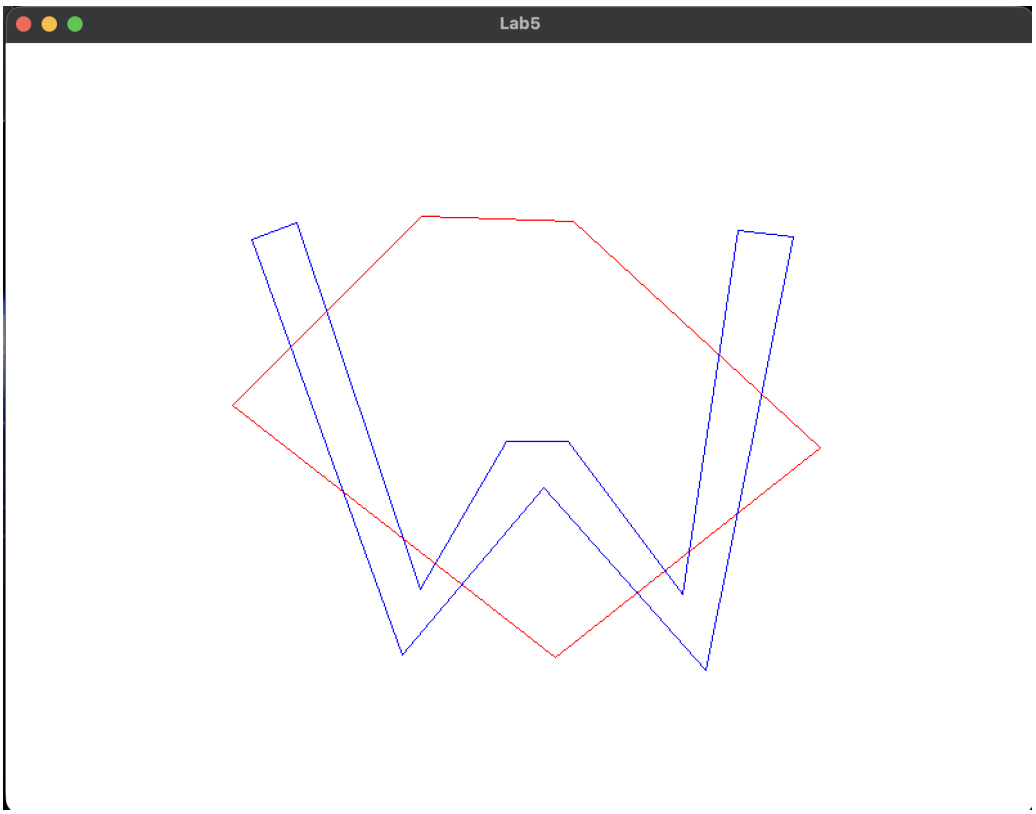


Рис. 1 — Результат работы

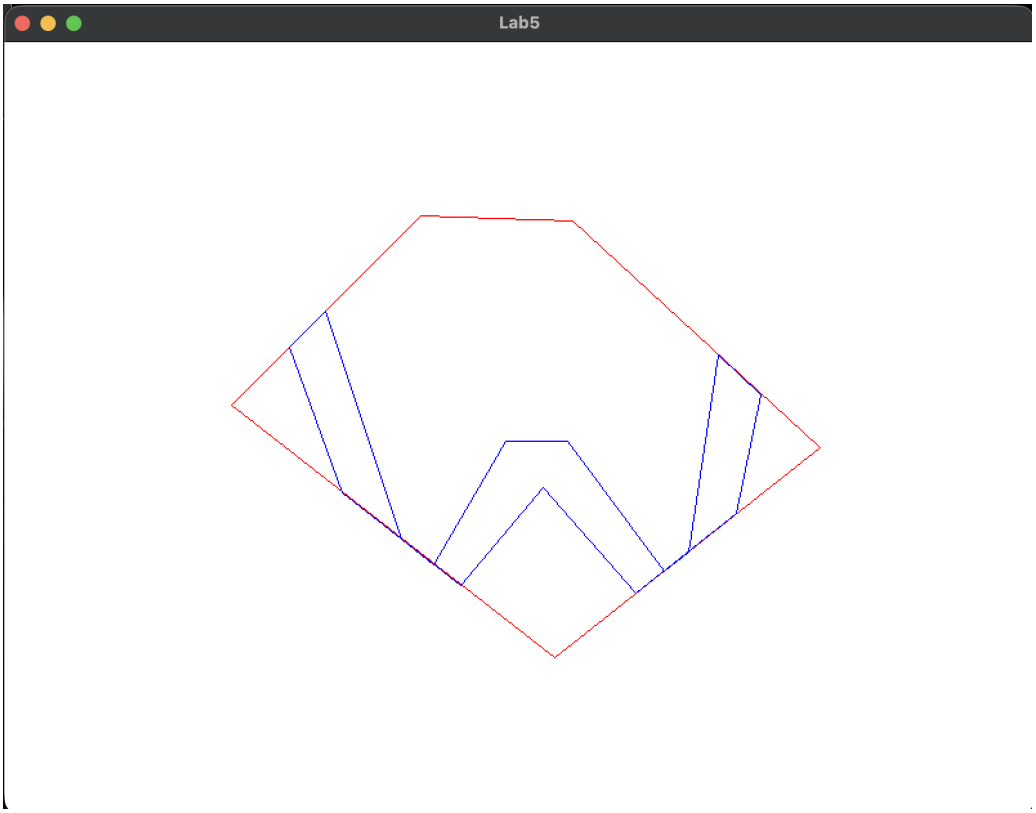


Рис. 2 — Результат работы

4 Вывод

В ходе лабораторной работы реализован классический алгоритм Сазерленда–Ходжмена для двумерного отсечения многоугольников. Программа позволяет интерактивно задавать вершины отсекателя и отсекаемого многоугольника, а затем вычислять их пересечение (внутреннюю область).